

T10.5 - Arquitectura de datos

Arquitectura de datos

Fully Incremental

Almacenar únicamente la *verdad más reciente*.

Al procesar un nuevo evento, se *actualiza la verdad y se descarta la historia*.

Problemas:

- No tolera errores humanos → no se puede corregir retrocediendo.
- No se puede operar sobre datos históricos → predicciones.
- En un *sistema distribuido* se debe *elegir entre consistencia y disponibilidad*.

Event Sourcing

Almacenar todos los eventos (inmutables) desde el inicio. Estos eventos se guardan y quedan así, son inmutables.

Hacer una consulta implica recorrer todos los eventos relevantes del pasado y calcular el resultado. También se pueden estimar momentos del futuro.

Problemas:

- Esto es *más complejo* → mantener ingreso de eventos sumado al cómputo que conlleva.
- *Mayor costo de almacenamiento* → se guardan eventos históricos ahora.
- Si la historia es grande, *la escala pasa a ser un problema*. Esto es porque saber la verdad actual (un saldo en una cuenta) implica recorrer todos sus eventos anteriores.

Consistencia

Contrato entre los procesos y el almacén de datos sobre la visibilidad y orden de las actualizaciones.

Son garantías mutuas: si los procesos respetan ciertas reglas (cómo y cuándo leen/escriben o mantenerse dentro del mismo server), el almacén promete un comportamiento determinado sobre qué van a ver cuando lean.

Los *niveles de consistencia* que puede ofrecer un sistema distribuido son:

- **Fuerte.** Toda lectura devuelve el último valor escrito.

Escribir `saldo = 100` implica que cualquier cliente en cualquier server lee al instante `saldo = 100`.

- **RYOW (Read Your Own Writes).** *El cliente ve sus propias escrituras inmediatamente, sin importar el server.* Las de otros clientes no necesariamente.

Escribir `saldo = 100` solo hace que uno mismo lea `saldo = 100` al instante. Otro cliente puede leer un valor viejo todavía. Uno ve sus propios cambios, los cambios de los otros pueden tardar.

- **De sesión.** *Como RYOW pero solo dentro de la misma sesión* (usualmente mismo server).

Sesión → conexión lógica continua entre un cliente y un server. Mientras esa conexión se mantenga, el sistema "sabe" que sos el mismo cliente hablando con el mismo nodo.

Igual que RYOW pero solo mientras te mantengas en la misma sesión (típicamente conectado al mismo server). Si cambiás de sesión o de server, perdés esa garantía.

- **Eventual.** Con el tiempo, *las lecturas convergen al último valor escrito.*

No hay ninguna garantía inmediata. Solo se promete que, si dejan de entrar escrituras, eventualmente todos los servers van a converger al mismo valor. Mientras tanto, podés leer datos desactualizados.

Teorema CAP

Es *imposible garantizar las tres propiedades al mismo tiempo*:

- **Consistency.** Todos los nodos ven los mismos datos al mismo tiempo.
- **Availability.** Si se puede comunicar con un nodo, se pueden leer/escribir datos.
- **Partition tolerance.** el cluster sobrevive a particiones de red.

Cluster → conjunto de máquinas o nodos que trabajan juntas como si fueran un único sistema.

Partición de red → cuando la comunicación entre nodos de un cluster se corta, pero los nodos siguen vivos. Quedan "islas" de nodos que no pueden hablar entre sí.

Como las particiones hay que tolerarlas (falacias de la computación distribuida), se elige entre CP o AP.